ROLLING-HORIZON CFDRP IMPLEMENTATION

MINOR PROJECT REPORT

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

(Department of Computer Science and Engineering)

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)



Submitted by

Aryan Jaiswal(24U020130)
Karan Sahu (24U020127)

Under the supervision of

Dr. Neeta Anna Eapen
Assistant Professor
(Department of Information Technology)

October, 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)



CERTIFICATE

This is to certify that the work embodied in this report entitled **"Rolling-horizon CFDRP implementation"** has been satisfactorily completed by Aryan Jaiswal(24U020130) and Karan Sahu(24U020127). It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Technology during the academic year 2026-27.

Date: 20th October,2026

Dr. Neeta Anna Eapen
Assistant Professor
Department of Computer Science and Engineering
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
Department Coordinator
Department of Computer Science and Engineering
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)



DECLARATION

We hereby declare that the following minor project report entitled "Rolling-Horizon CFDRP Implementation" presented here is in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology**. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Neeta Anna Eapen**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Aryan Jaiswal(24U020130)
Karan Sahu (24U020127)

ACKNOWLEDGEMENT

We, Aryan Jaiswal and Karan Sahu, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled "Rolling-Horizon CFDRP Implementation".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Prof. Dr. Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. His/her encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, Dr. Neeta Anna Eapen, for her constant guidance, invaluable suggestions, and expert advice throughout the project. His/her knowledge and insights helped us enhance the quality of our work, and her support kept us motivated during difficult times.

We would also like to express our gratitude to all the **faculties** of the Computer Science Department for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Karan Sahu (24U020127)
Aryan Jaiswal (24U020130)

AREA OF WORK

This project is about the transportation of small items by using Dynamic assigning orders to the drivers and we have implemented it and stored in the txt format for the performance analysis and simulation based performance analysis. In this the Dynamic Routing Problem called CFDRP (crowdsourced freight delivery routing problem) continuously orders comes and we updated our decision periodically called rolling horizon .

In this project we have implemented algorithmic and simulation routing framework based on the research paper using multi-day and CSV based datasets. Rolling horizon is use to assign drivers to the newly arrived orders and updates of delivered items and reroute the driver paths at decision epochs. Routing strategies from the research papers are used to evaluate the delivery schedules and costs.

The main aspect of the project is to transform into executable methods and csv based simulation environment. At each epoch the process is to allocate order , route evaluation and by the implementation it provide the practical framework to examine the dynamic delivery system behavior.

Overall, in the project it demonstrates how the algorithms of dynamic logistics optimization of the research have applied . It compares the performance evaluation and scalable simulation and the real-time routing visualization systems.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Area of Work	5
	List of Figures	7
	List of Tables	8
	Abstract	9
1	INTRODUCTION	10
2	LITERATURE REVIEW	13
3	PROBLEM DEFINITION AND OBJECTIVES	15
4	PROPOSED METHODOLOGY AND WORK DESCRIPTION	18
5	PROPOSED ALGORITHM	25
6	PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD	26
7	TOOLS, TECHNOLOGY AND IMPLEMENTATION DETAILS	31
8	RESULT DISCUSSION AND COMPARATIVE ANALYSIS	35
9	CONCLUSION AND FUTURE SCOPE	39
10	REFERENCES	41
11	PLAGIARISM REPORT	43

LIST OF FIGURES

Fig.	Description	Page no.
4.1	Order lifecycle and event-based delivery status logic	20
4.2	Conceptual pickup and delivery time-window feasibility timeline	21
6.1	Overall architecture of the CSV-driven CFDRP implementation	26
6.2	The schema and validation flow for reading input CSV files	27
6.3	Route diagrams	28
6.4	Lexicographic objective hierarchy used in the code	29
6.5	IPGA-style population search flow used by the implementation	29
6.6	Simulated annealing candidate-search process	30
8.1	Time-window feasibility by dataset, Days 1-10	37
8.2	Solver selected by epoch, Days 1-10	38

LIST OF TABLES

Table No	Description	Page no.
4.1	Required columns in the order and vehicle CSV files	19
7.2	Implementation differences and their meaning	33
8.1	Paired benchmark datasets and reduced run configuration	35
8.2	Outcomes for all ten matched datasets	37

ABSTRACT

Dynamic crowdsourced freight delivery requires vehicles to be assigned to paired pickup-and-delivery orders while order arrivals, vehicle positions, time windows, handling durations, and cargo loads change over time. This project implements a CSV-driven simulator for the Crowdsourced Freight Delivery Routing Problem (CFDRP), based on the rolling-horizon framing in Zhang [1]. At decision epoch one hour update, the orders seen at call-in are scheduled with the vehicle update simultaneously. A schedule of candidate orders is generated using a population search based on an Improved Parallel Genetic Algorithm (IPGA), then it is further improved with simulated annealing. In the experiment results, the order at the bottom of a schedule has the best lexicographic score, in general. Orders are prioritized with:

1. Time window conformity
2. The total lateness minimization
3. Estimated costs minimization.

The epoch snapshot (TXT), order history (CSV), four Comparators' implementation on each epoch are provided for a final audit. This procedure has been tested on 10 different data sets, with 2 orders for each match pair for D1-10, matched order/vehicle set 1,156 in total. The setting uses a population size of 16, five generations for a genetic algorithm, and limited 100 iterations for simulated annealing. The main seed 20261015 is used in the experiment. (1,316) orders in the selected plans have all been given some tasks and an estimated ETA. With 1,156 orders all (87.84%) follow the specified pickup or delivery time windows whereas 160 orders (12.16%) violate at least one constraint. These numbers indicate that the plans follow the CSV more closely, thus highlighting the auditable scheduling behavior that could be proved using validation based on delivery confirmation, global optimality or equivalence to the full implementation of the complete paper. The Euclidean distance has been used for estimating the path cross-order leg of each order due to the lack of the road network of the CSV data.

Keywords: crowdsourced logistics, dynamic vehicle routing, pick-up and delivery, CFDRP, rolling horizon, IPGA-style search, simulated annealing, time windows, reproducible simulation.

1. INTRODUCTION

1.1 Background

Two of the most challenging decision-making problems in last-mile delivery planning are routing [6], [7], [8] and delivery planning [4], [2], [3], [5], [6]. In these problems, route predictability must be balanced against the stochasticity of customer demand arrivals and the spatial distribution of the fleet throughout the day. As the number of customer demands increases and changes in the size of the fleet and task durations occur (which often are greater than predicted) the cost-effectiveness of static routing decreases [4]. Crowdshipping with additional vehicles and outsourced drivers potentially will only exacerbate the problem. Several of the recent published decision models have pointed out the value in simultaneously taking all assignment, pickup, delivery, travel, capacity, and service times into account [4], [2], [3], [5] however this often is not done with typical delivery models.

The CFDRP studied in the paper combines a pickup-and-delivery problem with a dynamic vehicle-routing problem. Orders are specified by call-in times, pickup and delivery locations, cargo volumes, projected travel times, service times, and service windows (a specified period of time in which the delivery is to be made), including early or late arrivals. Drivers send updates on their location and availability through vehicle update records. A route plan is created using the known data at some decision time and is updated as new data is received [1].

1.2 Why a Rolling-Horizon Simulator?

For delivery system, developing a full-day delivery plan faces difficult, because new orders are constantly coming in and vehicle data are changing during the whole working period. Rolling-horizon methodology overcomes this difficulty by decomposing the working period into time segments called decision epochs. At each epoch, the information at hand is used to develop a revised delivery plan, and then the cycle returns for the next decision epoch. It offers a feasible solution between having a fixed delivery plan and continuously re-optimizing delivery schedule in light of new information [4].

In this study, each decision epoch is set to one hour, following Zhang [1]. The program assumes only those orders with call-in times that have occurred are considered. Any deliveries completed would be entered into the epoch in which their completion is observed and would not be optimized in subsequent epochs, with remaining routes still assigned to their original vehicles, while

still out for delivery, while remaining orders could be ordered in the route of the assigned vehicle. Thus the simulator updates the delivery schedule for the assigned vehicle, while maintaining the vehicle's previous assignment. Presumably, this process could also continue between epochs as new orders and vehicle statuses are received.

1.3 Organization of the report

Chapter 2 covers the related work in crowdsourced delivery, pickup-and delivery routing, dynamic vehicle routing and heuristic optimization. Chapter 3 discusses the scope, problem, objectives, assumptions and notation of the project. Chapter 4 describes the workflow and time calculations. Chapter 5 explains the IPGA-style and SA algorithms.

Chapter 6 further illustrates the system and data-flow diagrams developed by me.

Chapters 7 and 8 The technologies and implementation of the project are covered.

2. LITERATURE REVIEW

2.1 Crowdsourced delivery and crowdshipping

Crowdsourced delivery involves non-dedicated drivers (not employed directly by a logistics company) to deliver an order between a specified pickup point and a customer destination. It raises a number of complex joint planning questions about drivers' supply, matching of orders, incentives, and the uncertainty of available vehicles. A survey by Alnaggar, Gzara, and Bookbinder discusses the presence of different platforms and the literature on research on the subject of crowdshipping. Savelsbergh and Ulmer describe the latest developments and trends, related challenges and opportunities, dynamic planning, and interactions between crowdsourced capacity and traditional delivery. Alnaggar, Gzara, and Bookbinder review the platforms and research literature and identify the many forms of crowdshipping models [5]. Savelsbergh and Ulmer discuss current operational challenges and opportunities, including dynamic planning and the relationship between crowdsourced capacity and traditional delivery operations [3].

Early work by Archetti, Savelsbergh, and Speranza studies vehicle routing with occasional drivers [6]. Macrina and colleagues extend occasional-driver routing with time windows [7]. Arslan et al. formulate crowdsourced delivery as a dynamic pickup-and-delivery problem with ad hoc drivers [8]. Dayarian and Savelsbergh examine same-day crowdshipping using in-store customers, while Qi et al. explore shared mobility for last-mile delivery and the resulting design and operational implications [9], [10]. These studies establish the importance of jointly considering delivery requirements and a changing driver supply.

The literature on recent crowdshipping research is concerned with the profile

of active crowdshippers [11], On the other hand, parameters of active crowdshippers are examined in three levels. While bathske and m nch analyze trips originating from occasional crowdshippers [11], Torres, Gendreau, and rei analyze crowdshipping with stochastic destinations [12], other studies include vehicle routing with stochastic supply and time windows [13]. However, Dai, He, and Ma are concerned with pricing of an on-demand platform with different levels of service [2]. These studies speculate that not only the distance, but also the availability, timing, customer commitments, and platforms' incentives, have impacts on the quality of routing.

2.2 Pickup-and-delivery problems and time windows

In a PDP, every request involves a pickup node and a delivery node, and therefore exhibits a precedence relationship of the two nodes as pickup before delivery. By time-window variations, both earliest and latest times are constrained, and with capacity variations the possible size of load that a vehicle is able to carry is limited. These constraints are expected to lead to a high degree of combinatorial complexity.

3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem statement

The input for the delivery plan sequence over the discrete decision epochs include an input list of orders and vehicle update orders. An individual order includes: call-in time, pickup node, delivery node, pickup window, delivery window, volume of cargo, service time, distance from merchant to customer, and vehicle travel time from merchant to customer. An individual vehicle includes: vehicle number, updating epoch, and position.The planner will probably plan for new input orders at the k-th epoch and replan unstarted pending orders with ongoing work continued. Furthermore, the simulator needs to output the approximated pickup event, delivery event, vehicle allocation, window state and final audit for each input order that can be scheduled.

3.2 Objectives

- The plan chosen must seek to minimize the number of orders which have any violations of the pickup or delivery time window.
- Among solutions with the same number of violations, consider the solution with the minimum total sum of pickup and delivery lateness.
- In addition, when solutions being considered are otherwise comparable, it is wise to seek to reduce the estimated vehicle fixed, travel, and lateness costs.
- When generating batches for pickups for the same merchant, we should respect the capacity limits, and the priority of pick-up should be higher than that of delivery.
- Re-optimizing at each epoch using only the orders and vehicle updates available at each decision epoch is the recommended approach.
- Additionally, creation of Audit ready TXT and CSV outputs is valuable to enable being able to see status changes and algorithms made.

3.3 Scope and exclusions

Implemented: CSV parsing, schema validation, one-hour-time-epochs, orders revealed flexibly, vehicle position updating, an IPGA-style candidate search, a simulated annealing alternative, schedule selection, time-window auditing, output. Not implemented: the underlying graph of road network, real traffic, real GPS streams, rejection of drivers, support of several different vehicle types with a queue to handle them, a separate exact MILP solver, actual field-based measurements of delivery completion.

3.4 Assumptions

- The order and vehicle CSV files' schemas confirm to the exact column names specified in the implementation provided.
- Additionally, the time columns are measured in hours since the beginning of the planning horizon; the epoch k should start at time hour k.

-Average velocity is kept constant at 30 km/hr, so an estimation of the duration is needed to go from one coordinate system to another. The maximum allowed car capacity is fixed at $7.2m^3$, so oversized individual orders will probably be rejected at the initialization step. Finally, the distances and times from merchant to customer are used to go from the origin to customer for the first step of a delivery from pickup batch and Euclidean distance is used for the following steps when no road information exists.

For this implementation, a schedule is called feasible if the event times calculated for all feasible orders in the current schedule have expected times of arrival (ETA) for pickup and delivery, respectively, and both completion events meet their corresponding late-window limits. The computational identifier indicates that the original instance likely does not have a better global solution when the event-clock heuristic classifies a schedule as INFEASIBLE. This label also indicates the current solution either does not satisfy the late-window requirement, or that the schedule selection does not satisfy the customer constraints.

-Average velocity is kept constant at 30 km/hr, so an estimation of the duration is needed to go from one coordinate system to another. The maximum allowed car capacity is fixed at $7.2m^3$, so oversized individual orders will probably be rejected at the initialization step. Finally, the distances and times from merchant to customer are used to go from the origin to customer for the first step of a delivery from pickup batch and Euclidean distance is used for the following steps when no road information exists.

For this implementation, a schedule is called feasible if the event times calculated for all feasible orders in the current schedule have expected times of arrival (ETA) for pickup and delivery, respectively, and both completion events meet their corresponding late-window limits. The computational identifier indicates that the original instance likely does not have a better global solution when the event-clock heuristic classifies a schedule as INFEASIBLE. This label also indicates the current solution either does not satisfy the late-window requirement, or that the schedule selection does not satisfy the customer constraints.

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Overall workflow

This flow begin with investigating the order and vehicle csv files. Validation before data is accepted is done by checking for required headers. Based on the existing literature, order id should have content and be unique therefore each file can require a minimum of 1 row of data. The vehicles are sorted by update epoch and then by vehicle number and the orders are sorted by call in time then order number. The order attribute value dictionaries and vehicle update dictionaries are created, travel-distance lookups are produced, compare the order volume with the capacity, create the objective weightings.

4.2 Input data and preprocessing

The order file requires 14 specific fields. The first two are record number and call-in time. Then four coordinate columns specify merchant and customer locations, then four columns specify early and late pickup and delivery times. Further fields include merchant-customer distance, travel time, goods volume, and loading/unloading time. The vehicle file requires four fields, vehicle number, update epoch, and horizontal and vertical coordinate fields. Each set of the paired local benchmark inputs is composed of 163, 167, 82, and 169

orders for Days 1, 2, 4, and 9, and 56, 57, 30, and 56 update rows in their corresponding vehicle files.

Table 4.1 shows the required CSV headers of the input validator. Presumably leaving the column headers unchanged allows the development of a runner that can accept new data and explains missing-field errors, rather than substituting data by default.

Table 4.1. Required columns in the order and vehicle CSV files

File / record	Required columns / purpose
Orders	Order number; unique, non-empty order ID
Orders	Call-in time; when the order becomes visible
Orders	Horizontal / vertical customer coordinates; delivery node
Orders	Horizontal / vertical merchant coordinates; pickup node
Orders	Pickup time window (early), Pickup time window (late)
Orders	Delivery time window (early), Delivery time window (late)
Orders	Distance between customer and merchant
Orders	Time consuming (h); source-supplied trip time
Orders	Volume of goods (m3); capacity check
Orders	Loading and unloading time (h); service duration
Vehicles	Vehicle number; vehicle identifier
Vehicles	Updating Epoch; position update boundary
Vehicles	Horizontal / vertical coordinate of vehicle node

4.3 Rolling-horizon execution

Simulates the time to k hours at each epoch k. Finishes all route events that are finished before starting new optimization. Loads vehicle update rows for the epoch k and updates all known vehicle positions. Starts with the START snapshot again because it was written to before the optimization so that the report can document the state of the pending orders at the decision boundary. Pending set is all orders calls-in before k and whose deliveries are not yet marked completed.

This implementation uses an epoch length of one hour. Visibility of an order is based on call-in time, not predicted information. As a result, an order will be invisible up until call-in time ≤ t_k, which is designed to separate the dynamic decision making from future arrivals.Among the different tasks at the boundary, the system is initially recording delivery events that are estimated to finish at this time. Next, the current update's rows are read, the currently visible set of tasks waiting to be executed is assembled and handled.

Moreover, the system records the selected assignment and logs the epoch audit. The boundary will probably run a separate continuation phase that could run all pending estimated time of arrivals (ETAs) after the last vehicle update, rather than scheduling one more update to those vehicles.

Two search procedures are carried out from the same visible pending set, current version, with a greedy seed and generates a population for an IPGA which may generate candidate assignment and order permutations. SA is also initialized with a greedy seed candidate and neighborhood moves. Usually the better candidate with lexicographic scores is selected. Then a committed solution is generated with assignment and assignment epoch, route lists are reordered, and MIDDLE and END snapshots are generated to show sequence of the scheduled ETA events within the epoch.

Figure 4.1 summarizes the event life cycle used by the simulator. The lifecycle starts by the order becoming ready after the call-in time. Next, it has been allocated to a driver and an estimated delivery and pickup time is set. An order is only considered to be delivered if its actual delivery event occurs. Any remaining order during the event will be in transit and not prematurely completed at the epoch boundary.

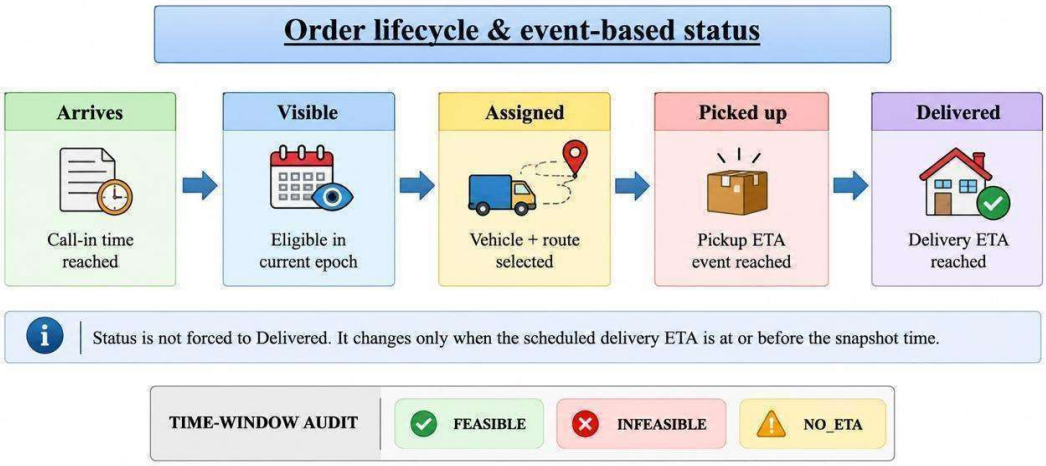


Figure 4.1. Order lifecycle and event-based delivery status logic.

4.4 Time and distance estimation

The segment (merchant -> customer) for a direct travel from merchant to customer is simply the distance and travel time corresponding to the order CSV already stored. (For other coordinate-to-coordinate segments, the code simply uses the Euclidean Distance.) Suppose each a is given in the

coordinate form (xa, ya), then the estimated distance is set to d(a, b):= sqrt((xa - xb)^2 + (ya - yb)^2). In addition, when travel time needs to be estimated based on a distance, we would guess that a fixed average speed of 30 km/h is used: travel_time(a, b):= d(a, b)/30. Figure 4.2. This figure shows early and late service limits and calculated event time. All distances in km, the time required to travel between points are given in hours. The data does not include an actual street graph and road-distance matrix which implies the segments are "as the crow flies" and will underestimate the total road distance and travel time in those lines. The routes schematic in Figure 6.3 are presented here only as an illustration and are not to be taken as an exact, turn-for-turn street map.

$d(a,b) = \text{sqrt}((x_a - x_b)^2 + (y_a - y_b)^2)$

The average travel speed used by the researchers' models, when calculating for average travel time over a long distance, was as follows: 30 km/hr.

$\text{Travel_time}(a, b) = d(a, b) / 30.$

This equation yields travel times in hours, where the distance is in kilometers. The data do not contain a detailed street graph, and does not contain a route matrix, and therefore the travel times are based on straight-line distance, which will be an underestimate of both distances and travel times. The route suggested in Figure 6.2 is simply a static schematic of the route.

Figure 4.2 shows the individual checking of early and late bounds at pickup and delivery. Existence of time-window failure is indicated here with a delivery otherwise given for Arrival Time, before an early boundary could lead to a delay, after a late one may cause a window fail.

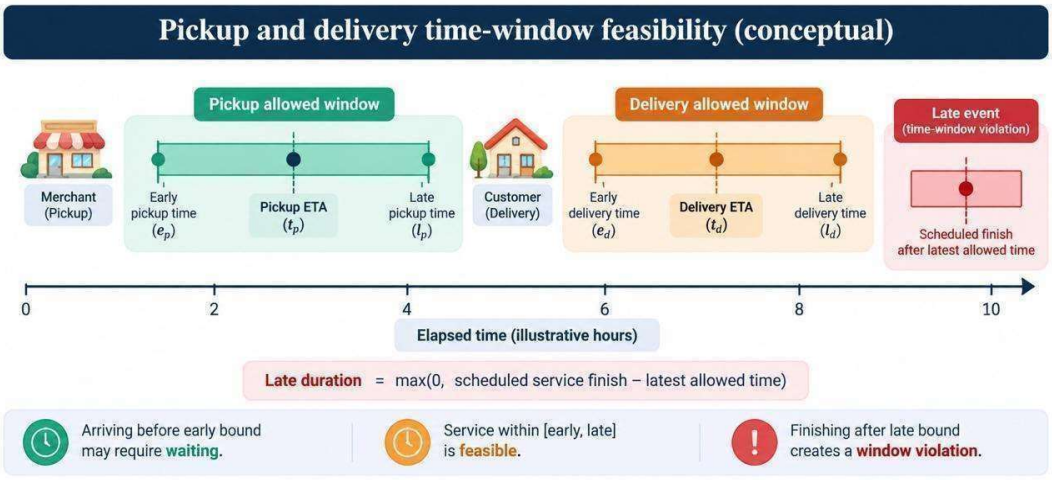


Figure 4.2. Conceptual pickup and delivery time-window feasibility timeline.

4.5 Pickup, delivery, service time, and lateness

The implementation splits the loading and unloading time of an order into two

halves: half before the order's pickup is complete and half after arriving at the place of delivery. Pickup completion can be no earlier than the arrival of the vehicle, the call-in time, or the early pickup bound; delivery start time can be no earlier than the customer's arrival or the early delivery bound. Studies indicate that the number of events in lateness is zero if the event ends at or before the late bound and one otherwise for the excess.

$pickup_start_i$ = max($vehicle_arrival_i$, $call_i$, $pickup_early_i$) $pickup_finish_i$= $pickup_start_i$+ $service_i$/ 2
$pickup_late_i$= max(0, $pickup_finish_i$- $pickup_late_bound_i$) $delivery_finish_i$= max($customer_arrival_i$, $delivery_early_i$) + $service_i$/ 2 $delivery_late_i$= max(0, $delivery_finish_i$- $delivery_late_bound_i$)
Furthermore, same-merchant batching can still involve multiple orders batched into a single batch for the merchant pickup if the info for the merchant matches and the total cargo volume does not exceed the volume threshold. It appears that the pickup service is ordered next and after that the ride is planned through all in a defined order. Additionally, the segment between the merchant and the customer is planning with source defined distance and time. Rest of the segments are planning with Euclidean distance.

4.6 Locked tasks and reassignable tasks

The live system is not expected to allow a job to be dispatched to a different vehicle if the current job is already underway – this may affect physical conditions. The live system also observes a protected prefix for each live route. It has been observed that jobs that have been dispatched are likely to continue as the leading vehicle on the route while any undispatched jobs may be inserted or rescheduled in the next epoch.This should allow the system to maintain service events for the assigned driver's vehicle as more customer vehicles come in. The record of history seems to contain the following types of information: assigned driver, assigned epoch, estimated time to pickup, estimated time to delivery, number of seconds since first call, how late at pickup and delivery, and state at each snapshot.

4.7 Objective function

The source paper defines a monetary target cost with fixed vehicle cost, travel costs and delay penalties [1]. The algorithm described in this paper uses a lexicographic method which suggests that schedules with the fewest orders causing violations of time-windows should be ranked before monetary costs. If Vbad is the total number of violations, Ltotal is total lateness of pickups and

deliveries in tenths of a minute and C_total is the total estimated dollar cost, then score as a function of weighted sum. Ordering between measures (Hierarchy of measures) 1. The number of violations is ranked higher than total aggregate lateness and total lateness is ranked higher than cost. This hierarchy is different from the cost-minimization goal of the paper which needs to be taken into account while evaluating the algorithm-selection scores. Score = W_v * V_{bad} + W_l * L_{total} + C_{total}

When evaluation candidate solutions, comparisons are made with respect to the same criteria and the candidate which has the lowest candidate score is chosen. The lexicographical scoring is intended to produce candidates with few time-window failures in order to minimize the number of trade-offs with respect to the small monetary savings obtained. From experiment, the preference is for candidates which have overall few orders affected in exchange for a marginally higher estimate of the distance cost during the project run, do not confuse this with the optimized quantity in Zhang [1].

4.8 Output and audit design

The approach assumed three output artifacts to improve from accessibility and reproducibility point of view. TXT report was expected to provide START, MIDDLE, and END tables of each epoch, comparison of algorithm per epoch, final delivery audit etc. CSV history table was expected to provide a snapshot-level order state. Pie chart may be added in further work. CSV algorithm table is expected to include the following: visible (pending) item count, new visible items, reassigned items, and available vehicles. The following were expected to be included in the algorithm table: scores of IPGA and SAs, selected solver, and run time. The audit was expected to include order information, ETA, window feasibility, violations, total pickup, total lateness, total delivery, total lateness, maximum delivery, maximum lateness, unassigned order id, and orders not having ETA.

Throughout the project, there are three different audits, which require a full sort. These audits are 1. Assignment coverage which is the assignment of order to vehicle,
2. Scheduled completion which begins the order schedule with first est delivery time, and 3. Time-window coverage which requires the order to be picked up and delivered within its time window. An order can go through two audits and still be unable to meet the third audit.

In addition, the dashboard HTML page can also serve as a live visualiser and logs

activity from the instant the new CSV is submitted onto the web page controls. The very first implementation deploys a humble greedy playback planner and does not make use of the Python IPGA/SA optimization. The quantitative processing outlined in this report makes use of the Python runner and does not, by intention, deploy the browser animation function as an optimization which is part of the Python runner.

5. PROPOSED ALGORITHM

5.1 High-level rolling-horizon algorithm

The inputs of this algorithm are three files ORDERS-Csv (O), Vehicles Update CSV (U) and Search Parameters. They are used to produce a schedule, a time-window audit and reproducibility artefacts. The algorithm builds the schedule an order schema, some identifiers, initial vehicle state, route plan and history of delivery events in CSV format. At each decision epoch k (kth vehicle update epoch), the time is forwarded until hour k and at each decision epoch the following steps are executed: it generally accepts the deliveries with ETA estimated, and apply the kth vehicle position update. It then determines the set of visible pending orders Pk and secures route prefixes for active and in-progress orders. An IPGA-style search on candidate space is then performed to greedily optimize ETAs. Simulated annealing with greedy seed is used, the nth vehicle position update is applied, the simulated ETAs are run, and the following products are then created: an audit (TXT), history (CSV) and algorithms comparison (CSV).

5.2 IPGA-style population search

The IPGA style candidate search is used to return a greedy schedule seed. Some of the order variant value sets can change at some factors without effecting route fitness from one candidate search to another. It is proposed that the candidate search is ordered so that the top 2 candidates are likely to survive through the elitism. Candidates are generated with rank based weights. Order variants are generated using the order permutation operator, corresponding to the seed, parent and current candidate. Assigned choices are then based on previous solutions. Locked order assignments are used and the route-change operator is used to generate the candidate. Return the best candidate of each final generation. The IPGA style candidate search flow diagram in Chapter 6 attempts to explain the code to this level of detail without requiring to duplicate the code for the MATLAB chromosome or copy the whole MILP model from the paper.

6. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

6.1 System architecture

Figure 6.1 is the architecture overview for the implemented pipeline: orders/vehicles CSVs pass through schema validation and state initialization; the epoch-specific pending set is sent to the IPGA-style and SA searches; the lower selected score determines the committed plan; and snapshot/audit writers persist the outputs. Future orders are not passed to the search before their call-in times become visible.

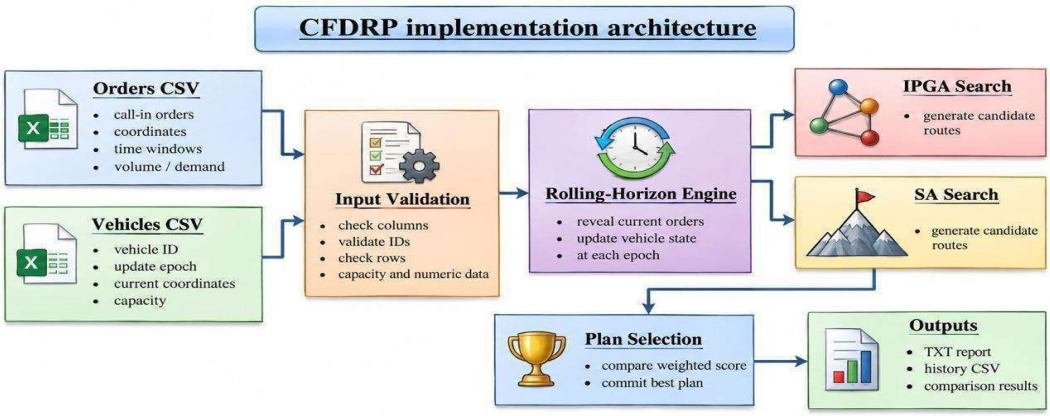


Figure 6.1. Overall architecture of the CSV-driven CFDRP implementation.

6.2 Data-flow diagram

Figure 6.2 shows the input contract before the import process is optimized. The parser will require an order and vehicle column and is probably to decline input that is entirely empty and lacks one or more order and vehicle columns. It will also decline input with repeating order IDs, input that is missing a vehicle key, and input that has orders that go beyond the modeled vehicle capacity.

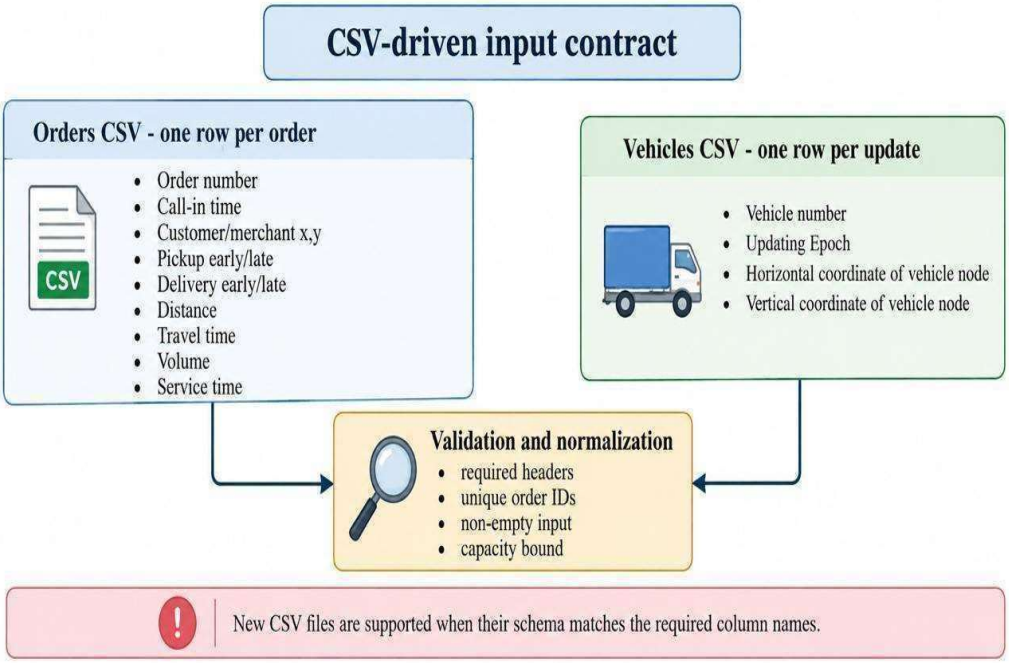


Figure 6.2. The schema and validation flow for reading input CSV files

Order file and vehicle file ordering in the sequence submitted to the validator are as shown in 6.2. Order properties are probably created after validation and vehicle update groups are created by epoch. Only the current visible pending set, vehicle and route state are supplied to the optimizer. Snapshots rows and algo level records are supplied to the report writer. This should aid in debugging input errors and scheduling decisions away from output formatting errors.

6.3 Time-window and route diagrams

The event-state diagram explains the terms employed in describing a terminal output: The label In transit is used until the ETA arrives and the implementation does not assume that a late order would be late. The time window status is stored separately, i.e., in addition to an ETA an order can be labeled as INFEASIBLE if it does not meet either pickup or delivery window.

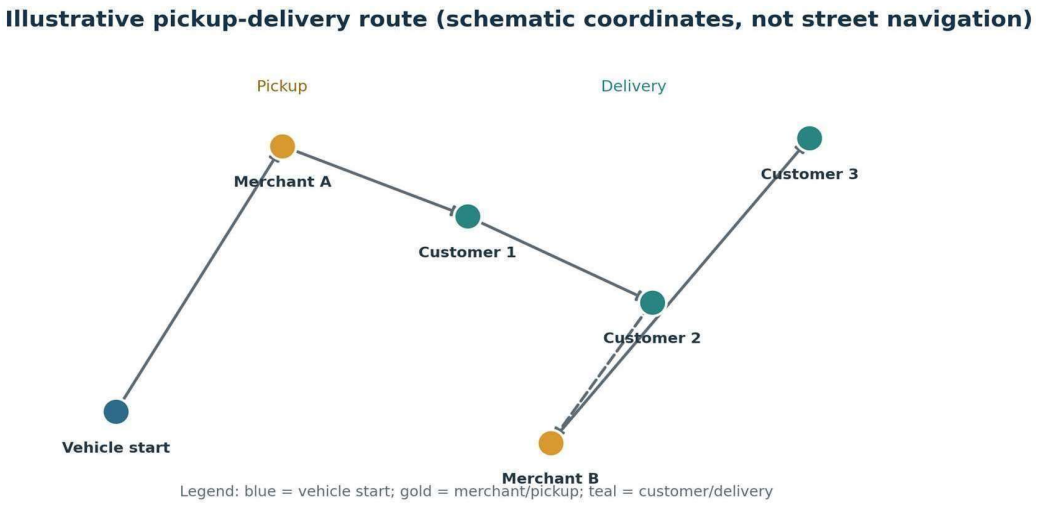


Figure 6.3. Route Diagrams

Figure 6.3: Higher-level diagram of visitation order in one typical case of a merchant pickup node coming before customer delivery node. In this diagram, a schematic diagram is depicted on an (x, y) coordinate plane for simplicity because there is not a street graph in the CSVs provided, cross-node travel has been assumed by Euclidean distance in the implementation. These figures are meant to provide a conceptual diagram to illustrate the basic ideas of the implemented model. These figures are not based on any paper. Location coordinates in the route diagram are conceptual, and are not intended to match any specific street travel patterns

6.4 Lexicographic objective hierarchy

Figure 6.4 shows the complete hierarchies of goal bases used in the coding of hierarchies. The ordering of these hierarchies is probably logical: the total late minutes come after the number of late orders.



Figure 6.4. Lexicographic objective hierarchy used in the code.

6.5 IPGA-style population search

The population search loop that defined the process adopted by the project. The process can be summarized through the following main steps: greedy initialization, candidate ranking, elitist hiring, route-order alterations, evaluation and restart.

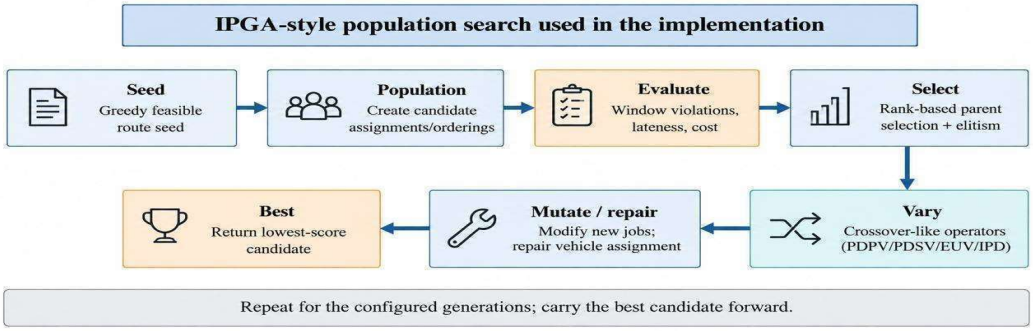


Figure 6.5. IPGA-style population search flow used by the implementation.

6.6 Simulated annealing comparison

In the simulated annealing algorithm, a neighboring candidate is tested and accepted if it is an improvement. Candidate solutions that are not an improvement are occasionally accepted if they meet the temperature rule.

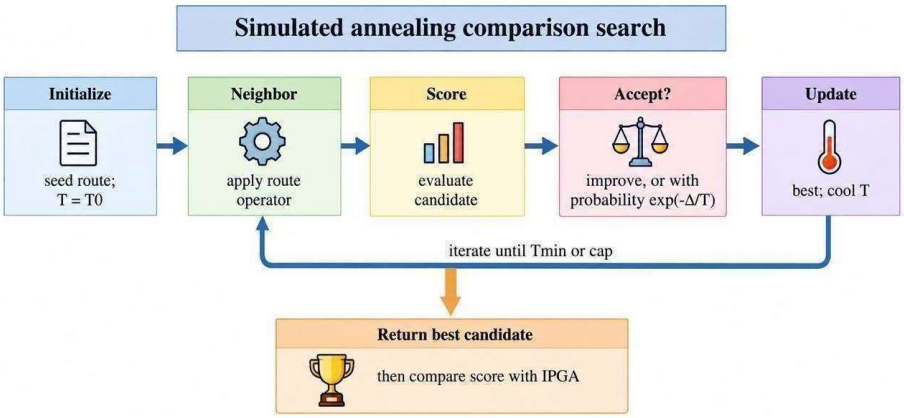


Figure 6.6. Simulated annealing candidate-search process.

7. TOOLS, TECHNOLOGY AND IMPLEMENTATION DETAILS

7.1 Python

Python is used for the end-to-end simulation because it supports CSV handling, optimization loops, structured output, and repeatable command-line execution. The submitted code uses standard modules such as argparse, csv, math, random, statistics, time, data classes, pathlib, and typing. Randomness is controlled by an explicit seed so that a run can be repeated with the same dataset and parameter settings, subject to the same Python/runtime behavior.

7.2 pandas and CSV I/O

pandas loads order and vehicle records from CSV files and provides sorting and row-level data access. The program checks that all required column names exist before simulation starts. It rejects missing input files, empty files, duplicate or blank order identifiers, blank vehicle IDs/epochs, and orders larger than the nominal single-vehicle capacity. The file names are provided through command-line arguments, so a new compatible dataset does not need to be hard-coded into the source.

7.3 Command-line execution

The script is run from a terminal window. A sample run using the Day 1 pair with changed paths to a different matched pair of CSVs An implementation of header validation and produces: (a) a text epoch report (b) a row-wise history CSV (c) an algorithm-comparison CSV twin file The script runs as follows:

``

```
python3 cfdrp_simulator.py \

--orders day01orders.csv \

--vehicles day01vehicles.csv \

--report day01epochoutput.txt \

--history day01epochhistory.csv

--population 16 --generations 5 --sa-iterations 100
```

``

The local benchmark is run on a smaller setting, population=16, generation=5, and SA cap=100 for these are not too expensive for experiments to be rerun. It should be noted that the default for the script is population=500, generation=500, and SA cap=10000 and these were not used in the numerical tables of the report; so most numerical statements, expect claimed sources, should be made on the reduced setting. The exceptions are of course with claimed sources where they are given or data from source papers.

7.4 Development environment

The code is running out-of-the-box on any normal Python3 environment and in any operating system (macOS, Linux, Windows) if pandas is installed. The reproductions are indicating the command line arguments, CSV file paths and seed used in the output logs. It is recommanded to also save the checksum of the script or the commit id, the version of pandas and python, the specifications of the hardware and the context.

7.5 Snapshot generation and final audit

It is foreseen that the START, MIDDLE and END images will be created for each epoch and components on the image is expected to include the following; visibility limit, pickup/delivery point location, call in time, driver identifier, assignment epoch, expected time of arrival (pickup/delivery) in order, call in time (in minutes), lateness indicator 1/2, time window status, event status and the model phase. Additionally, in order to make it possible to see the transition of those finished orders from the order set, completed orders could be placed in its completion epoch in all epoch images for at least one snapshot period until deleted at the END epoch. Besides, it was understood from the recent audit that order number is likely unnecessary, because order completion will involve event time (e.g., event time in a plan).

7.6 Example output interpretation

The report contains, between the '- - - - -' lines, the following entries: "ASSIGNED DRIVER=V17", "PICKUP ETA= 04:32", and "DELIVERY ETA= 05:18". The pickup/ delivery estimated times are 0:0 in the HH:MM format. This entry on the report implies that, for the presently selected schedule, the ETA's for pickup and delivery are estimated in elapsed time rather than in the last time window in which the pickup/ delivery will occur. The status 'FEASIBLE' indicates that neither the pickup or delivery late-window is being violated by this schedule. If the status reads 'INFEASIBLE' then one or the other is being violated by this schedule; it does not necessarily mean that any schedule with this total time is infeasible (but also does not necessarily mean that it is feasible!) "NO_ETA" indicates that the present schedule has failed to generate the delivery event for the order in question. This should be taken into account in compiling the results with a project guide.

7.7 Design decisions that differ from the research article

8.1 Table of measures scale between the prototype and the paper The measure differences exist, which are relevant for the feasibility and run-time as the current route description and trip estimation are a simplified version of the original, full-fledged optimization model.

Table 7.2. Implementation differences and their meaning

Aspect	Reference paper [1]	Supplied implementation in this project
Mathematical model	MILP formulation for each rolling-horizon subproblem	Heuristic candidate evaluation; no exact MILP solver is invoked
Algorithm	IPGA and SA as described in the article	Independent IPGA-style search and SA comparison in Python

Objective priority	Minimize monetary service cost with delay penalty	Lexicographic: number of violating orders, total lateness ticks, monetary cost
Travel model	Model-specific real data/network assumptions	Source merchant-customer leg plus Euclidean cross-order legs
Route representation	Paired pickup/delivery node structure	Restricted order-level route sequence and same-merchant batching
Driver/route logic	Paper-specific assignment and route constraints	Started route prefix locked; unstarted jobs may be re-ordered
Validation	Article's own experimental protocol	CSV schema validation and final schedule audit
Interpretation	Paper-reported RMB cost savings and case study	Simulated ETAs; no GPS/field validation or optimality proof

8. RESULT DISCUSSION AND COMPARATIVE ANALYSIS

8.1 Benchmark protocol

The current benchmark is based on all ten matched order/vehicle input pairs, shown in the following: Day 1 (163 orders, 56 vehicles, 6 epochs); Day 2 (167 orders, 57 vehicles, 6 epochs); Day 3 (82 orders, 30 vehicles, 3 epochs); Day 4 (82 orders, 30 vehicles, 3 epochs); Day 5 (77 orders, 29 vehicles, 3 epochs); Day 6 (163 orders, 56 vehicles, 6 epochs); Day 7 (76 orders, 29 vehicles, 3 epochs); Day 8 (168 orders, 57 vehicles, 6 epochs); Day 9 (169 orders, 56 vehicles, 6 epochs); Day 10 (169 orders, 56 vehicles, 6 epochs). The current experimental scale uses a population of 16, 5 generations of improved particle GA (IPGA), 100 maximum iterations of simulated annealing (SA), and

Table 8.1. Paired benchmark datasets and reduced run configuration

Item	Setting / dataset	Interpretation
Paired input days	Day 1 through Day 10 (10 matched pairs)	Each day has both order and vehicle CSV inputs
Orders / vehicles / epochs	163/56/6, 167/57/6, 82/30/3, 82/30/3, 77/29/3, 163/56/6, 76/29/3, 168/57/6, 169/56/6, 169/56/6	Order rows / unique vehicle IDs / update epochs in day order
Epoch interval	1 hour	Vehicle Updating Epoch defines the decision boundary
Primary parameters	Population=16, generations=5, SA cap=100	Reduced reproducibility
Primary seed	20261015	All ten primary per-day results use this seed
Sensitivity seeds	20261015, 20261016, 20261017	Three-seed subset remains Days 1, 2, 4 and 9 only
Route distance	Euclidean for unsupported legs	CSV files do not provide a full road graph / road-distance matrix
Objective	Feasibility, then lateness, then estimated cost	Weighted lexicographic score; not currency
Recorded outputs	TXT report, history CSV, comparison CSV	Individual per-day outputs and combined 10-day summary included
Update epochs across study	48 decision epochs	3 epochs on Days 3, 4, 5 and 7; 6 epochs on other days
Input completeness	All ten days paired	No dataset excluded from the primary day-level benchmark

20261015 as the seed for the primary. This reduced representation of the numerical experiments is intended to experiment with data flow and allow for multiple repetitions, not the full-scale 500 member/500 generation design described in the paper. Additionally, the three seed sensitivities for the previous report are only presented for days 1, 2, 4, and 9, but 6 additional tests are run for one primary seed with a single search budget. Table 8.1 summarizes all ten matched order/vehicle pairings with the consistent reduced scale. There are no days intentionally omitted from these comparisons.

8.2 Dataset-level outcomes

All ten combinations of schedule were, for each source order, all 1,316 source orders were assigned to drivers, and the delivery estimated times of arrival (ETAs) were determined according to the appropriate scheduling system. Based on an independent audit of time windows, 87.84% (1,156 out of 1,316) of orders were found to be compliant, while 12.16% (160 out of 1,316) of orders were likely to violate at least one pickup or delivery late boundary. Day 5 had the lowest proportion of compliant orders, 76.6% (59 out of 77), while Day 8 had the highest proportion of compliant orders, 93.5% (157 out of 168). Note that "scheduled completion" is simply an estimated ETA at the time of simulation, not a GPS-confirmed or on-the-ground delivery result. Figure 8.1 shows the assignment coverage and time-window compliance across all ten days, while Table 8.2 shows the same data for each individual day for the purposes of examining assignment coverage and time-window compliance. Note that these results differ due to the fact that an order could have an ETA calculated yet still violate a pickup or delivery late boundary.

Table 8.2. Outcomes for all ten matched datasets.

Dataset	Orders	Vehicles / epochs	Assigned	Compliant (%) / violations	Last ETA
Day 1	163	56 / 6	163 / 163	149 (91.4%) / 14	09:55
Day 2	167	57 / 6	167 / 167	148 (88.6%) / 19	09:53
Day 3	82	30 / 3	82 / 82	63 (76.8%) / 19	07:59
Day 4	82	30 / 3	82 / 82	63 (76.8%) / 19	07:59
Day 5	77	29 / 3	77 / 77	59 (76.6%) / 18	07:45
Day 6	163	56 / 6	163 / 163	149 (91.4%) / 14	09:55
Day 7	76	29 / 3	76 / 76	60 (79.0%) / 16	07:50
Day 8	168	57 / 6	168 / 168	157 (93.5%) / 11	09:30
Day 9	169	56 / 6	169 / 169	155 (91.7%) / 14	10:52
Day 10	169	56 / 6	169 / 169	153 (90.5%) / 16	10:59
All ten	1316	—	1316 / 1316	1156 (87.84%) / 160	—

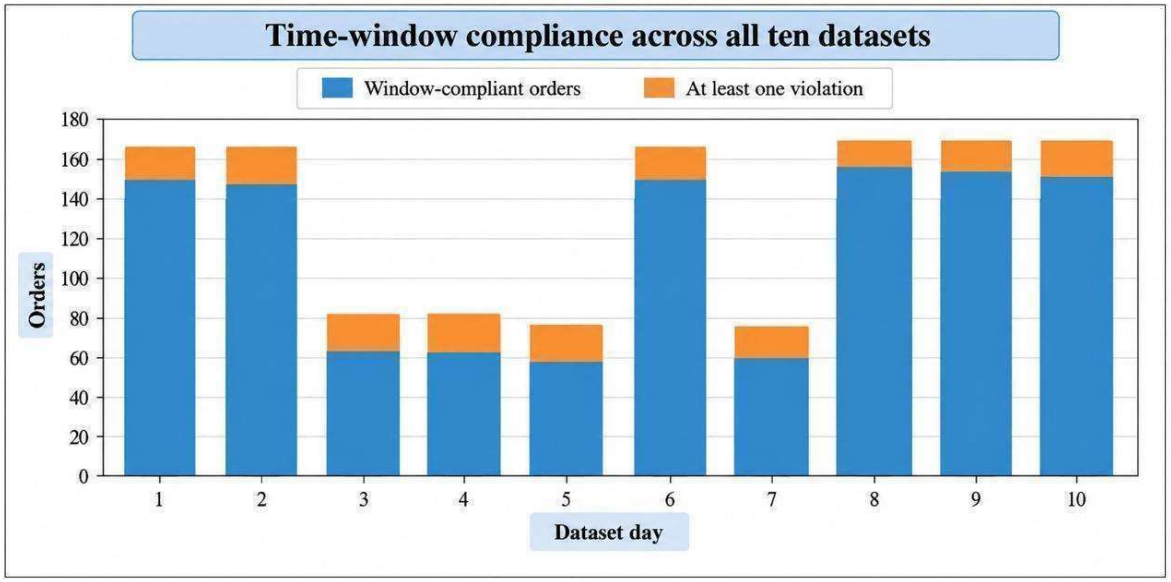


Figure 8.1. Time-window feasibility by dataset, Days 1-10.

8.3 Solver selection and runtime

For each of the chosen epochs, the simulator runs an IPGA-style search, as well as a SA style search, on the same specified lexicographical objective and records the schedule with the lower score. The four main solvers are used, in order of the primary solution score, for 10 runs, representing the wins by each solver on each day and epoch; the counts are provided in Table 8.4 and are not indicative of any particular solver selection. The primary measurement is lateness, with secondary importance given to feasibility and monetary costs. Due to the tight search budget and stochastic nature of candidate moves, these results are likely noise.

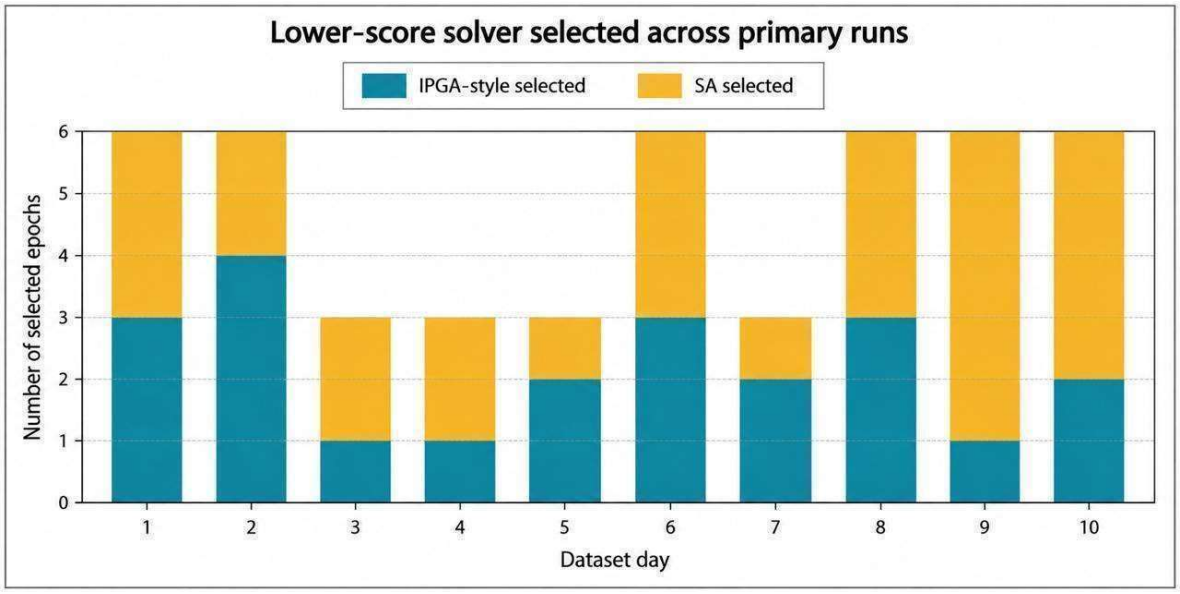


Figure 8.2. Solver selected by epoch, Days 1-10.

The per-epoch run times of both searches are both kept in the same respective CSVs that contain the rest of the algorithm comparison data. In Figure 8.6, we plot the total timestamp for the jobs from these totals for all 10 datasets. These totals do not include initialization, parsing, report formatting, and file writing. In addition, these totals represent local times at low parameter values and cannot be expected to provide guarantees about deployment latency or asymptotic efficiency

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The project reported here is a CSV file driven simulator that is used to tackle the Crowd Source Freight Delivery Routing Problem (CFDRP). The simulator receives order updates and vehicle updates, maintains a state, and produces candidate schedules using an IPGA style search and simulated annealing search. The simulator takes the candidate schedules and chooses the one with the lowest lexicographic score, then records an audit of the chosen schedule. The report displays the presence of any time window violations that occurred. It does not hide any violations behind the number of orders completed.

For the primary reduced-parameter simulation conducted on all 10 matched data-sets, all 1,316 source orders were assigned a primary driver and provided with a scheduled delivery window ETA. Of these 1,316 orders, 1,156 (87.84%) remained within the pickup and delivery late-window constraints, and 160 orders departed from the late-window constraints. The largest observed delivery-window constraint violation was 328.0 minutes (Day 9). The three-seed sensitivity analysis is currently limited to days 1 to 10. However, with the six days recently added, primary-seed results are consistent with the full data-set results and an indication of future candidate days for a repeated-seed sensitivity analysis. Again, it should be pointed out that these results are scheduled simulations and are not field-delivery records.

The implementation here does not exactly reproduce the model shown in the paper, having a simpler approximation of the routes, a Euclidean approximation for parts of a journey where no distance is specified, smaller test sets, and a lexicographical goal with first-feasibility. This implementation is not an exact reproduction of the authors' implementation in MATLAB, nor their complete written form of the MILP model. Therefore, the cost savings given in the paper are given separately, and the local results are seen as validation of the functional prototype implementation rather than direct evidence for how likely the implementation is to be able to drive costs down to the level shown in the paper.

9.2 Future scope

In the future, we could use a road network like distance scheme and not this Euclidean distance, which is a straight line distance for calculating the estimates. And handling new data.We could also do this 500 membered, 500 generation IPGA configuration and repeat the same experiment with the new seeds, which are random before having a final conclusion.Use different

routing policies and compare all the different policies for monetary benefit or feasibility. Calculate the different objective weights and penalty formulations and report both of the results of feasibility-first and cost-first search separately. We could also optimize the load carrying and use maximum out of the given vehicle load, and also load vehicle after every event for more general multi order carrying. We can integrate stress tests for repeated vehicle ID, oversize orders, late order arrivals, and no vehicle epochs.

10. REFERENCES

[1] J. Zhang, “Pickup and delivery planning for the crowdsourced freight delivery routing problem,” PLoS ONE, vol. 20, no. 2, Art. no. e0318432, Feb. 2025, doi: 10.1371/journal.pone.0318432.

[2] Y. Dai, S. He, and Z. Ma, “Pricing decisions for an on-demand crowdsourced delivery platform with differentiated service levels,” IEEE Transactions on Engineering Management, vol. 71, pp. 14350-14364, 2024, doi: 10.1109/TEM.2024.3450574.

[3] M. Savelsbergh and M. W. Ulmer, “Challenges and opportunities in crowdsourced delivery planning and operations-an update,” Annals of Operations Research, vol. 343, pp. 639-661, 2024, doi: 10.1007/s10479-024-06249-1.

[4] L. Glomb, F. Liers, and F. Rosel, “A rolling-horizon approach for multi-period optimization,” European Journal of Operational Research, vol. 300, no. 1, pp. 189-206, 2022, doi: 10.1016/j.ejor.2021.07.043.

[5] A. Alnaggar, F. Gzara, and J. H. Bookbinder, “Crowdsourced delivery: a review of platforms and academic literature,” Omega, vol. 98, Art. no. 102139, 2021, doi: 10.1016/j.omega.2019.102139.

[6] C. Archetti, M. Savelsbergh, and M. G. Speranza, “The vehicle routing problem with occasional drivers,” European Journal of Operational Research, vol. 254, no. 2,
pp. 472-480, 2016, doi: 10.1016/j.ejor.2016.03.049.

[7] G. Macrina, L. Di Puglia Pugliese, F. Guerriero, and D. Laganà, “The vehicle routing problem with occasional drivers and time windows,” in Optimization and Decision Science: Methodologies and Applications, Springer Proceedings in Mathematics & Statistics, vol. 217, pp. 577–587, 2017, doi: 10.1007/978-3-319-67308-0_58.

[8] A. M. Arslan, N. Agatz, L. Kroon, and R. Zuidwijk, “Crowdsourced delivery-a dynamic pickup and delivery problem with ad hoc drivers,” Transportation Science, vol. 53, no. 1, pp. 222-235, 2019, doi: 10.1287/trsc.2017.0803.

[9] I. Dayarian and M. Savelsbergh, “Crowdshipping and same-day delivery: employing in-store customers to deliver online orders,” Production and Operations Management, vol. 29, no. 9, pp. 2153-2174, 2020, doi: 10.1111/poms.13219.

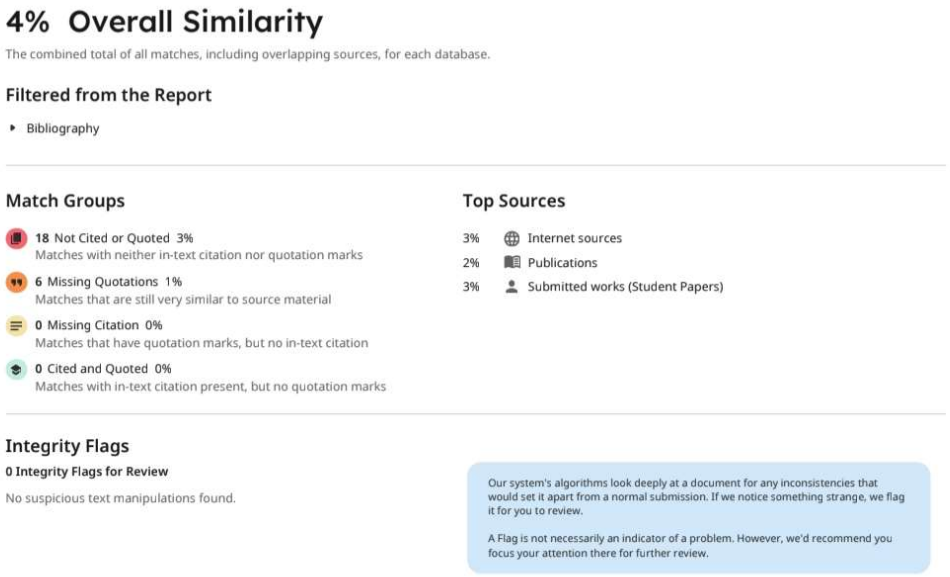
[10] W. Qi, L. Li, S. Liu, and Z. J. Shen, “Shared mobility for last-mile delivery: design, operational prescriptions, and environmental impact,” Manufacturing & Service Operations Management, vol. 20, no. 4, pp. 737-

751, 2018, doi: 10.1287/msom.2017.0683.
[11] H. Bathke and C. Münch, “From occasional to active crowdshippers: the significance of couriers’ characteristics,” IEEE Transactions on Engineering Management, vol. 71, pp. 12094-12109, 2024, doi: 10.1109/TEM.2023.3244422.
[12] F. Torres, M. Gendreau, and W. Rei, “Crowdshipping: An open VRP variant with stochastic destinations,” Transportation Research Part C: Emerging Technologies, vol. 140, Art. no. 103677, 2022, doi: 10.1016/j.trc.2022.103677.
[13] F. Torres, M. Gendreau, and W. Rei, “Vehicle routing with stochastic supply of crowd vehicles and time windows,” Transportation Science, vol. 56, no.

11. PLAGIARISM REPORT

The project report was checked using Turnitin.

The overall similarity index is 4%, with bibliography matches excluded. No integrity flags were detected.



The AI detection result is below Turnitin's 20% reporting threshold; therefore, no exact percentage is displayed

